

SMART INDIA HACKATHON 2026

Problem Statement 26153 · National Technical Research Organisation (NTRO) · Blockchain & Cybersecurity

CyberOracle

AI-Based Network Attack Forecasting from Network Traffic Data

A Transformer–GNN World Model for Proactive Cyber Defence

MASTER RESEARCH, ARCHITECTURE & EXECUTION DOSSIER

Prepared for Team Precognix

Idea-Submission Round — Online PPT, Prototype & Video Evaluation

Submission deadline (per SIH portal): 20 September 2026

Compiled 5 September 2026

How To Use This Document

This dossier is the single reference for winning Problem Statement 26153. It has three jobs:

- (1) prove to you and to the evaluators that the team understands the problem at a depth beyond the PPT bullet points.
- (2) give you a concrete, buildable architecture with an honest account of what is prototype-stage today versus what the official brief actually requires for this round.
- (3) give you a day-by-day plan to the **20 September 2026** submission deadline, plus a large bank of rehearsed answers for the viva or any evaluator questions.

MOST IMPORTANT FINDING — READ THIS FIRST

The official PS 26153 brief on the SIH portal asks for more than a PPT with links. The listed deliverables for evaluation are:

- (1) a source code link (GitHub/Drive)
- (2) a README with setup instructions
- (3) an Architecture Document of at most 2 pages
- (4) a demo video of at most 2 minutes
- (5) a technical presentation of at most 5 slides.

Section 1.3 breaks this down in full, and the Appendix contains copy-ready drafts for the Architecture Document and the README so nothing is missing on submission day.

Sections are written to be read in order once, then used as standalone references afterwards — jump straight to Part 10 for the build plan, Part 11 before a mock viva, or the Appendix the night before submission. Headings use Word's built-in heading styles, so the Navigation Pane (View → Navigation Pane) will let you jump between sections instantly.

A note on rigour: every factual claim about the problem statement, the sponsoring institutions, the datasets and the cited research below was verified against live sources in September 2026 rather than recalled from memory, because a hackathon submission that quotes a stale statistic or a mis-cited paper loses credibility fast in front of a technical panel. Sources are listed in Part 12.

Contents

Part	What's inside
Part 1	Decoding Problem Statement 26153 — what NTRO is actually asking for
Part 2	Why This Problem Matters — India's threat landscape and the institutions behind the PS
Part 3	The Science Behind CyberOracle — world models, graph learning and the 2025–26 literature
Part 4	The CyberOracle Architecture — formal design, component by component
Part 5	Frontend ↔ Backend — exactly how data moves through the system
Part 6	Who Benefits — stakeholders, user personas and deployment sites
Part 7	Why CyberOracle Wins — novelty and competitive positioning
Part 8	Feasibility, Viability and Risk
Part 9	Impact and Benefits
Part 10	Step-by-Step Build Plan — day-by-day to 20 September, then the 36-hour Grand Finale plan
Part 11	The Full Viva & Judge Q&A Bank
Part 12	Glossary and References
Appendix	One-page cheat sheet, PPT tightening notes, copy-ready Architecture Document and README

Part 1 — Decoding Problem Statement 26153

Before any architecture or slide gets built, the team has to be able to restate this problem statement more precisely than the organisation that wrote it. This section reconstructs PS 26153 from the live SIH portal listing (captured 4–5 September 2026) and breaks it into what is being asked, what is merely suggested, and what is silently mandatory.

1.1 Snapshot

Field	Value
Problem Statement ID	26153 (SIH26153)
Title	AI based Network Attack Forecasting from Network Traffic Data
Sponsoring organisation	National Technical Research Organisation (NTRO)
Theme	Blockchain & Cybersecurity
Category	Software
Submission deadline	20 September 2026 (idea-submission / online round)
Dataset guidance given by SIH	nciiipc.gov.in for context; CIC-IDS2017/2018, UNSW-NB15, CTU-13, CICIoT2023, LANL Authentication Dataset, DARPA intrusion datasets, plus MITRE ATT&CK, CAPEC and CVE/NVD as knowledge bases
Central technical concept named by NTRO	“World Models” — explicitly named in the brief, not just implied

Two things matter immediately. First, NTRO did not ask for “an intrusion detector” — the brief explicitly frames the ask around *World Models*, a specific and fairly academic AI concept (Part 3 gives it a full treatment). A team that cannot explain what a world model is, and why it differs from a classifier, has not answered the question that was actually asked, however good their dashboard looks. Second, the dataset list already tells the panel exactly which benchmark numbers they expect to see quoted back at them — producing a result on a dataset that is not on this list, without also covering one that is, reads as not having read the brief.

1.2 What NTRO Is Actually Asking For

Stripped of its framing language, the brief asks for a system that does five things, in this order:

1. Turn raw network telemetry (flow records and/or packet captures) into a structured **network state** at each point in time — either a feature vector or a graph.
2. Learn how that state evolves — a model of the transition probability from the current state to the next one, not a lookup table of known attack signatures.

3. Run that learned model forward for several steps (a “K-step rollout”) to produce a forecast: a probability that the network is heading toward compromise, before it gets there.
4. Translate that forecast into something a defender already understands — a MITRE ATT&CK stage (Reconnaissance, Initial Access, Lateral Movement, Command & Control, or Exfiltration).
5. Explain the forecast — name the specific ports, flags or flow statistics that are driving the number — using attention weights, SHAP values, or an equivalent feature-attribution method.

Every one of these five is graded, and the brief is unusually explicit that step 5 is non-negotiable: an opaque score with no explanation is called out as *unacceptable*, not merely “nice to have.” This lines up exactly with the architecture Team Precognix has already been sketching (Graph State Constructor → GNN Spatial Encoder → Transformer World Model → K-step rollout → SHAP/attention → ATT&CK mapping) — the job in the next 25 days is to make each of those five boxes real, in that order of priority, rather than polishing the dashboard first.

The one sentence that defines the whole problem statement

An infiltration is a process unfolding over time, not a single anomalous packet — which is why treating each network flow in isolation, the way a conventional benign/malicious classifier does, throws away exactly the information (the order ports are probed in, the timing between reconnaissance packets, the sequence of SYN flags before an ACK flood) that would let a defender see the attack coming.

That single idea — sequence and timing carry information that a single-flow snapshot destroys — is the thesis every slide, every answer in the viva, and every design decision in the codebase should trace back to. If a judge asks “why not just use Random Forest on these features,” the honest, on-message answer is this sentence, not a generic “deep learning is more powerful” line.

1.3 The Exact Deliverables — What Actually Gets Submitted

This is worth re-reading twice

The brief lists five required artefacts for evaluation, and only one of them is the slide deck. Treat the other four as equally graded, not as optional extras that a good PPT can compensate for.

#	Deliverable	Hard limit	What it actually needs to contain
1	Source code link	GitHub or Drive link	A real, runnable repository — not a zip attached to the PPT. Public GitHub is strongly preferable: it is checkable with one click and signals confidence.
2	README	—	Setup instructions a stranger could follow: environment, dependencies, how to run training, how to run inference/demo, and what dataset to point it at. See the Appendix for a ready-to-adapt skeleton.

#	Deliverable	Hard limit	What it actually needs to contain
3	Architecture Document	Max 2 pages	A dense, diagram-led technical brief — not a rehash of the PPT. The Appendix contains a copy-ready draft in this exact 2-page shape.
4	Demo video	Max 2 minutes	A screen recording of the working demo interface actually ingesting a file and producing a forecast — two minutes is not enough time for a slow intro; open on the running tool.
5	Technical presentation	Max 5 slides	The standard SIH idea-submission template (Proposed Solution, Technical Approach, Feasibility & Viability, Impact & Benefits, Research & References) plus the Title Page — which matches the six-slide deck already drafted.

The message in this table is blunt: the **Architecture Document and README are new work**, not by-products of the PPT. A team that nails the slides and the video but submits a bare GitHub link with no README, or a 6-page “architecture document” that is really the whole research proposal, has technically failed to follow instructions on two of the five graded items — an easy way to lose marks that has nothing to do with the quality of the idea.

Provenance note: *this deliverables list and the background/description text it is drawn from were retrieved from a structured, regularly-updated mirror of the sih.gov.in problem-statement portal (captured 4 September 2026), cross-checked against the official portal's own published Software-track submission workflow. Portal content can be revised by NTRO at any time before the deadline, so the team should do one final read of the PS on sih.gov.in itself in the 24 hours before submitting — treat this document as the detailed briefing, not as a replacement for that final check.*

1.4 Reading Between the Lines — What Else Is Being Graded

The brief also contains several quieter requirements that are easy to miss on a first read but that an evaluator with a cybersecurity or ML background will specifically look for:

- **“Not a static classifier.”** The brief repeats this phrase twice. If the demo can be explained as “we trained a model to output benign/malicious,” that is the failure mode NTRO is explicitly screening out. The pitch and the code both need to foreground the transition-dynamics / rollout behaviour — showing the risk curve advancing across future time-windows, not just a single current-flow verdict.
- **“Generalise to unseen attack patterns — not merely memorise signatures.”** This is a direct invitation to run (and show) a generalisation test: train on some attack families, hold one out, and report what happens. Section 4.6 gives a concrete recipe for this that takes an afternoon and produces one of the most convincing slides in the whole deck.
- **The mandatory logistic-regression baseline.** The expected solution explicitly asks for benchmark numbers against “a logistic regression baseline trained on the same features,” with F1, precision,

recall and false-positive rate. This is not a suggestion — skipping it means the world model's central claim (that temporal dynamics help) is asserted rather than demonstrated.

- **“Fully offline, no cloud API dependencies.”** This is a hint about the intended deployment context (air-gapped government and CII networks — see Part 2) as much as a technical constraint. A demo that calls an external LLM API for anything, however small, contradicts the brief's own framing and should be avoided even for convenience features.
- **Two levels of features are required together, not either/or.** Flow-level (aggregate) features alone will miss a slow, low-and-slow reconnaissance scan designed to stay under flow-based thresholds; packet-level (timing/sequencing) features alone lose the aggregate picture of something like a SYN flood. The brief is explicit that both are required for exactly this reason — an evaluator may well ask the team to justify a design that only used one.
- **“The approaches are examples, not mandatory.”** NTRO explicitly leaves the exact architecture (LSTM vs Transformer vs GNN vs “other AI techniques”) open. This is an invitation to justify the Transformer+GNN combination on its own merits (Part 3) rather than because the brief told the team to use it — judges will likely probe why, specifically, over the listed alternatives.

1.5 The Datasets NTRO Explicitly Sanctioned

Using a dataset from this list — and citing it correctly — is itself a small trust signal to evaluators. Full technical detail on each is in Part 3.5; this is the quick-reference version.

Dataset	What it captures	Why it's on this PS specifically
CIC-IDS2017 / CSE-CIC-IDS2018	Canadian Institute for Cybersecurity flow + PCAP captures; brute-force, DoS/DDoS, web attack, infiltration, botnet scenarios with attack-window timestamps	The de-facto standard NIDS benchmark; has labelled attack timelines, which is exactly what the PS needs for supervised transition-dynamics training
UNSW-NB15	ACCS-simulated traffic; 49 features across 9 attack families (Fuzzers, DoS, Exploits, Reconnaissance, Worms, etc.)	A second, structurally different benchmark for the generalisation test the brief asks for
CTU-13	13 real botnet captures (Neris, Rbot, Virut, Sogou, Murlo, NSIS.ay, Menti) plus verified normal and background traffic, from CTU University, 2011	Real (not simulated) botnet C2 behaviour — strong for the Command & Control ATT&CK stage specifically
CICIoT2023	46.6M-record IoT testbed capture: DDoS, DoS, Recon, Spoofing, Mirai, web-based, brute-force	Newest dataset on the list; signals to evaluators the team looked past the well-worn 2017/2018 sets

Dataset	What it captures	Why it's on this PS specifically
LANL Comprehensive Multi-Source Cyber-Security Events	Los Alamos National Lab internal network logs (auth, DNS, process) with labelled red-team activity	Strong for authentication-log-driven lateral-movement and credential-access modelling, a different telemetry angle than flow/PCAP
DARPA intrusion datasets (incl. OpTC / Transparent Computing lineage)	Host + network provenance data, red-team-injected malware against baseline activity	The same family of data used by the most recent (2026) academic work on this exact problem — see Part 3.4

For the submission itself, the pragmatic choice — and the one the team's own technical-approach slide already leans toward — is to build primarily on **CSE-CIC-IDS2018** (flow CSVs are clean, well-documented and directly loadable) and use **CTU-13** as the held-out, structurally different dataset for the cross-dataset generalisation claim. UNSW-NB15 is a reasonable second cross-dataset option if time allows a third data point.

1.6 The Official 5W1H, Restated Cleanly

	Answer, in one line
What	A software prototype that learns a network's evolving state from traffic telemetry and forecasts the likelihood and stage of an in-progress attack before compromise completes.
Why	Because per-flow classifiers discard the sequencing and timing information that is the actual signature of an infiltration in progress — they can only confirm a breach after it has already happened.
Who	Sponsored by NTRO; primary institutional beneficiary is NCIIPC and the Critical Information Infrastructure sectors it protects; end users are enterprise and CII Security Operations Centre (SOC) analysts.
When	Operates continuously on rolling observation windows (seconds to tens of seconds) and forecasts K steps ahead (minutes), aiming to buy defenders lead time measured in minutes, not the days it currently takes to notice a slow intrusion.
Where	Deployed on-premises, passively, off a mirror/TAP port on an enterprise or CII network — explicitly required to run fully offline with no cloud dependency.
How	Dual-level feature extraction → graph/state construction → learned transition-dynamics model (Transformer/LSTM/GNN) → K-step rollout → MITRE ATT&CK stage mapping → SHAP/attention explanation → offline dashboard.

Part 2 — Why This Problem Matters

A judging panel for a national-security-adjacent problem statement will expect the team to know the institutional and threat landscape at least as well as the technical one. This section is the material for the first 60 seconds of the pitch and for any “why should we care” question.

2.1 India's Cyber-Threat Trajectory, In Numbers

CERT-In (the Indian Computer Emergency Response Team, the national incident-response agency under Section 70B of the IT Act 2000) publishes the closest thing India has to an official incident count. The trend is not gently rising — it is accelerating:

Year	Cyber security incidents tracked by CERT-In
2020	11,58,208
2021	14,02,809
2022	13,91,457
2023	15,92,917
2024	20,41,360
2025	29,44,000+ (over 29.44 lakh)

That is roughly a **2.5× increase in five years**, with the sharpest single jump between 2024 and 2025. In 2025 alone, CERT-In additionally issued 1,530 alerts, 390 vulnerability notes and 65 advisories, and ran 122 cybersecurity drills covering 1,570 organisations across defence, paramilitary, space and atomic-energy sectors — a scale of activity that only makes sense if the underlying threat volume genuinely justifies it.

Two 2026 data points sharpen the picture further for this specific problem statement. First, a national cyber-resilience report published in August 2026 recorded roughly **200,000 attempted intrusions against the power sector alone** within the assessment period — a single Critical Information Infrastructure sector, not the whole economy. Second, the same report documented a surge of about 1.5 million cyberattack attempts (DDoS, phishing, malware, defacement) coinciding with Operation Sindoor, India's May 2025 cross-border military operation, of which roughly 150 reportedly resulted in a successful compromise. Whatever one's view of the underlying event, the pattern it illustrates is exactly the one this problem statement is built around: attack volume against Indian networks spikes sharply and quickly around geopolitical flashpoints, and a defence posture that only reacts once a breach is confirmed is structurally too slow for that kind of surge.

2.2 The Institutions Behind This PS — Know Them Cold

NTRO — the problem sponsor

The National Technical Research Organisation is India's technical-intelligence agency, established in 2004, reporting to the National Security Advisor and the Prime Minister's Office. It is not a civilian software vendor — it is a technical-intelligence body, which explains the brief's emphasis on offline operation, explainability (analysts must be able to trust and audit a forecast, not just receive it) and generalisation to novel attacks rather than known signatures.

NCIIPC — the strategic beneficiary, and the dataset link NTRO pointed to

The brief's own dataset guidance names `nciipc.gov.in` first, before the public ML datasets. The National Critical Information Infrastructure Protection Centre is a unit of NTRO, established on 16 January 2014 under Section 70A of the Information Technology Act, 2000 (as amended in 2008), and is India's national nodal agency for protecting Critical Information Infrastructure (CII) — defined in the Act as computer resources whose incapacitation would have a debilitating effect on national security, the economy, public health or safety. NCIIPC has publicly identified six Critical Sectors, and CyberOracle's “applicability to enterprise and CII environments” claim should be understood against this exact list:

- Power & Energy
- Banking, Financial Services & Insurance (BFSI)
- Telecom
- Transport
- Government
- Strategic & Public Enterprises

NCIIPC's own stated functions — issuing guidelines and vulnerability advisories, running audits, funding R&D for “innovative future technology” for CII protection, and exchanging incident intelligence with CERT-In — map almost one-to-one onto what a successful CyberOracle deployment would feed into: audited, explainable early-warning intelligence that a nodal agency can act on across sectors, not just inside one company's SOC.

CERT-In — the national response layer CyberOracle would feed

CERT-In runs the Cyber Swachhta Kendra (botnet-cleaning service covering 98% of India's digital population), the National Cyber Coordination Centre (NCCC, metadata-level cyberspace monitoring for situational awareness), and a network of sectoral CSIRTs including CSIRT-Fin for the financial sector. CyberOracle's forecast-plus-explanation output is designed to sit one layer below this: a SOC-level early-warning feed whose aggregated signal is exactly the kind of situational-awareness input the NCCC model is built to consume.

2.3 Why Reactive Detection Keeps Losing — With Real Cases

The problem statement's core technical claim — that treating flows in isolation throws away the timing and sequencing signature of an in-progress attack — is not abstract. It is the pattern behind several of India's most consequential recent incidents:

- **AIIMS Delhi, November–December 2022.** A ransomware attack took down outpatient, inpatient, billing, lab and appointment systems for close to two weeks and reportedly exposed data linked to

tens of millions of patient records, with investigations spanning CERT-In, the NIA, Delhi Police and the CBI. The pattern in nearly every serious ransomware incident is the same: initial access and lateral movement unfold quietly over days before the encryption payload — the loud, obvious event — finally fires. A system that only alerts once payload execution starts has already lost the window that mattered.

- **Power-sector targeting, Mumbai (2020) and near Ladakh (reported 2022).** Independent threat-intelligence research has linked intrusion activity against Indian power-sector entities to state-linked groups active during periods of heightened border tension — exactly the kind of low-and-slow reconnaissance-then-lateral-movement pattern that flow-isolated classifiers are least equipped to catch early.
- **The broader ransomware baseline.** CERT-In's own India Ransomware Report has recorded a 51% year-on-year increase in ransomware incidents across sectors including critical infrastructure, and independent industry surveys have repeatedly placed India among the most-targeted countries globally for ransomware.

The throughline across all three: by the time a SIEM's rule fires or an analyst opens a ticket, the attacker is usually already well past reconnaissance. CyberOracle's proposition is to move the intervention point from “after the loud event” to “during the quiet build-up” — which is precisely the temporal window that per-flow classification cannot see.

2.4 The Policy Backdrop

The idea also sits inside an active compliance and policy push, which is directly useful for the Feasibility & Viability slide:

- **Section 70A, IT Act 2000 (amended 2008)** is the legal basis for NCIIPC's CII-protection mandate — the same section the SIH portal's own dataset note points to.
- **CERT-In's June 2023 information-security guidelines** for government entities explicitly cover network security, security monitoring and incident management — a predictive layer like CyberOracle is additive to, not a replacement for, this compliance baseline.
- **CERT-In's Security Assurance Framework** already empanels 231 certified audit organisations focused heavily on banking, finance, power and transport — the same sectors named as NCIIPC's Critical Sectors — giving a plausible, existing channel through which a tool like this could eventually be validated and adopted rather than needing to invent a new one.

Part 3 — The Science Behind CyberOracle

This is the part of the dossier to have open during the viva. Every claim here is traceable to a specific, named piece of research, which matters because “we used a Transformer because it's state of the art” collapses under one follow-up question, while “we used a Transformer because paper X showed it out-performs baseline Y on exactly this task” does not.

3.1 World Models — the theoretical core NTRO explicitly named

The term “world model” is not generic AI marketing language in this brief — it refers to a specific lineage of research. The foundational formulation is Ha & Schmidhuber's 2018 paper “*World Models*” (arXiv:1803.10122, presented at NeurIPS 2018), which splits an agent into three components:

Component	Role in the original paper	CyberOracle's equivalent
V — Vision / encoder	A Variational Autoencoder compresses each raw observation into a small latent vector	Graph State Constructor + GNN Spatial Encoder: compress a time-window of flow/packet features into a per-window graph embedding
M — Memory / transition model	A recurrent network (an LSTM with a mixture-density output) predicts how the latent state evolves over time	The Temporal Transformer (or LSTM, in the fallback build) that learns $P(S_{t+1}^{\square} \mid S_t^{\square}, S_{t-1}^{\square}, \dots)$
C — Controller	A small policy network that acts on the compressed state	The K-step rollout head plus the MITRE ATT&CK stage classifier — the component that turns the learned state into a decision-relevant output

The line worth quoting directly in the pitch is the paper's own framing: an agent that has “a good representation of past and present states, and a good predictive model of the future” can act before an event fully resolves, rather than only reacting to it. Two follow-on developments matter for justifying architecture choices further:

- **Hafner et al.'s Dreamer / RSSM line (2019 onward)** replaced the pure LSTM memory with a Recurrent State-Space Model that carries both a deterministic and a stochastic component, and later work (“Mastering Diverse Domains through World Models,” 2023) showed the same recipe scaling across very different environments — supporting the brief's own requirement that the model generalise rather than memorise a single dataset's signatures.
- **Yann LeCun's 2022 JEPA proposal** argued that predictive world models, not pattern-matching classifiers, are the more principled route to a system that can reason about futures it hasn't directly seen — the same argument, transplanted to network security, is exactly PS 26153's design philosophy.

The one-line answer to “why a world model and not just a better classifier”

A classifier answers “*is this flow bad?*” using only the present. A world model answers “*given how this network's state has been moving, where is it heading?*” — which is the only framing that can, in principle, produce a warning before the bad event has already happened.

3.2 Why Represent a Network as a Graph

Network flows are natively relational: a NetFlow record is a source–destination pair, i.e. an edge between two host nodes, carrying edge-level attributes (ports, flags, bytes, duration). Treating them as flat feature rows throws that relational structure away. The University of Queensland line of work formalises the fix:

- **E-GraphSAGE** (Lo, Layeghy, Sarhan, Gallagher & Portmann) extends the GraphSAGE algorithm to generate *edge embeddings* rather than node embeddings — the right unit of analysis when the traffic itself, not the hosts, is what is labelled.
- **Anomal-E** (Caville, Lo, Layeghy & Portmann, Knowledge-Based Systems, 2022) builds on E-GraphSAGE with a self-supervised training signal (a revised Deep Graph Infomax objective that corrupts edge features and trains the encoder to tell real from corrupted graphs), specifically to reduce dependence on labelled attack data — relevant background for CyberOracle's own generalisation argument.
- **GraphIDS** (Guerra et al., 2025) pairs an inductive E-GraphSAGE encoder with a Transformer-based masked autoencoder in one end-to-end architecture, reporting up to 99.98% PR-AUC and 99.61% macro-F1 across several NetFlow benchmarks — direct evidence that the GNN + Transformer combination CyberOracle already proposes is not a hackathon guess but a currently-competitive architecture family.

Practical translation for the build: model each observation window as a graph where **hosts (IP addresses) are nodes** and **flows are edges carrying the flow-level and packet-level feature vector**. A GATv2 or GraphSAGE layer aggregates each host's local connectivity pattern into a per-node embedding every window; pooling those into a single per-window graph embedding is what feeds the temporal model in Part 4.

3.3 Choosing the Temporal Engine: LSTM vs Transformer vs Temporal-GNN

Approach	Strength	Weakness	Verdict for this PS
LSTM / GRU over per-window embeddings	Simple, fast to train, small data needs, easy to explain to a non-ML judge	Struggles with very long-range dependencies (a reconnaissance scan hours before the payload)	Ship this first — it is the honest, low-risk baseline the team's own risk-mitigation plan already proposes

Approach	Strength	Weakness	Verdict for this PS
Temporal Transformer (self-attention over the window sequence)	Attention weights are directly usable as an explainability signal; better long-range modelling	More data-hungry; more hyperparameters to tune in limited time	The stretch target once the LSTM baseline works; matches the brief's explicit mention of attention-based explainability
Temporal Graph Network (TGN-style: memory module + GNN + time-aware attention)	State of the art on exactly this task per 2026 literature (see 3.4)	Most complex to implement correctly in a short timeframe; higher risk of subtle bugs	Cite as the architecture's natural extension point in the Research & References slide, even if not fully implemented for this round

3.4 The Live 2025–26 Research Frontier — Cite These, Correctly

Three very recent papers sit almost exactly on top of this problem statement. Being able to name them accurately, with what they actually did, is a strong differentiator — most teams will only find the older, more famous papers.

StageFinder — Phan & Bauschert, 2026 (accepted, IEEE ICC 2026)

“Learning the APT Kill Chain: Temporal Reasoning over Provenance Data for Attack Stage Estimation” (arXiv:2603.07560). Encodes host/network provenance graphs with a GNN to capture structural dependencies between processes, files and connections, then uses an LSTM to learn the temporal dynamics and estimate the current MITRE ATT&CK stage. Pre-trained on the DARPA OpTC dataset and fine-tuned on DARPA Transparent Computing data. Reports a macro-F1 of 0.96 and a 31% reduction in prediction volatility versus prior baselines. This is close to a direct precedent for CyberOracle's own GNN-plus-sequence-model-plus-ATT&CK-mapping design, on a DARPA dataset that is itself on NTRO's approved list.

BiTA — bidirectional GRU–Transformer temporal graph aggregator, 2026

“BiTA: Bidirectional Gated Recurrent Unit-Transformer Aggregator in a Temporal Graph Network Framework for Alert Prediction in Computer Networks” (arXiv:2604.22781) frames alert prediction explicitly as a temporal-graph-network problem and benchmarks against classical ML (SVM, Random Forest), static GNNs (GCN, GraphSAGE) and other temporal-graph models (DyRep, JODIE). It is useful less for its exact architecture and more as evidence that “alert / attack-progression prediction” is an active 2026 research category with its own emerging benchmark suite — worth a one-line mention in the references slide to show currency.

GraphIDS and the Anomal-E / E-GraphSAGE lineage — 2022–2025

Already described in 3.2 — the direct technical justification for representing traffic as edge-attributed graphs rather than flat feature tables, and for self-supervised or hybrid training when labelled attack data is scarce, which is realistically the case for any team working in a few weeks with public benchmark data.

3.5 The Sanctioned Datasets — Full Technical Reference

Dataset	Scale	Key attack coverage	Known limitations to mention proactively
CSE-CIC-IDS2018 / CIC-IDS2017	CIC-IDS2017: ~2.8M flow records, 80 CICFlowMeter features, 5 days, 25 simulated users, ~48.8GB	Brute-force, (D)DoS, web attack, infiltration, botnet, port scan, Heartbleed	Some known NaN/Infinity artefacts in pre-computed flow features; no encrypted-APT or IoT-protocol traffic
UNSW-NB15	175,341 train / 82,332 test records (from ~2.5M raw); 49 features via Argus & Bro/Zeek across 5 feature groups	9 attack classes: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms	Higher null-error rate than older KDD-style sets; normal traffic is not itself a live capture
CTU-13	13 scenario captures, ~76.8M malicious packets + 3.6M benign NetFlow records; CC-BY licensed	Seven real botnet families: Neris, Rbot, Virut, Menti, Murlo, NSIS.ay, Sogou	Botnet-specific — best used as the cross-dataset generalisation check, not the primary training set
CICIoT2023	46.68M total records across 79 PCAPs, 99GB+, 31+ hours	DDoS (72.8% of records), DoS, Mirai, Reconnaissance, Spoofing, web-based, brute-force	Extreme class imbalance toward DDoS — needs explicit class-weighting if used for training
LANL Comprehensive Multi-Source Cyber-Security Events	Multi-source logs (auth, DNS, process, flow) from LANL's internal network with labelled red-team events	Credential access, lateral movement, authentication-based attack chains	Host/auth-centric rather than packet-centric — a good stress test for whether the model generalises beyond pure NetFlow
DARPA intrusion datasets (LLDOS legacy; OpTC / Transparent Computing lineage)	OpTC: ~1,000 hosts, baseline activity plus red-team-injected malware	Full host+network provenance chains — the same family used by StageFinder (3.4)	Older LLDOS captures are dated; prefer OpTC/Transparent Computing subsets if time allows

3.6 Explainability — SHAP and Attention, and Why the Brief Insists on Both

SHAP (SHapley Additive exPlanations, Lundberg & Lee, NeurIPS 2017) borrows the Shapley value from cooperative game theory to fairly distribute a prediction's outcome across its input features — answering “how much did this specific feature push the score up or down,” with the guarantee that the individual contributions sum exactly to the difference between the model's output and its baseline. Attention weights, by contrast, are a by-product of the Transformer's own architecture — they show *which past time-windows or which graph nodes* the model weighted most heavily in producing its output, which is a different (and complementary) kind of explanation: SHAP explains *feature* importance for a single prediction; attention explains *temporal/structural* importance across the sequence or graph.

Both are worth implementing because the brief accepts either but a judge will likely ask why the team picked what it picked: the honest answer is that attention comes essentially free from the Transformer/GNN layers already in the architecture, while SHAP (via TreeSHAP or KernelSHAP against the K-step rollout head) gives the sharper, feature-level story (“SYN-flag variance contributed +38% to this forecast”) that a SOC analyst can act on directly.

3.7 The Knowledge Bases: MITRE ATT&CK, CAPEC, CVE/NVD

MITRE ATT&CK is the adversary-behaviour taxonomy at the centre of the whole design. As of the April 2026 release (**v19**), the Enterprise matrix has **15 tactics** after the framework's biggest recent structural change — splitting the old “Defense Evasion” tactic into separate *Stealth* and *Defense Impairment* tactics — alongside 222 techniques and 475 sub-techniques. PS 26153 itself asks only for the five coarse, classic stages (Reconnaissance → Initial Access → Lateral Movement → Command & Control → Exfiltration), which is the right scope for a hackathon build; being able to mention the v19 update in the viva signals the team tracks the framework rather than having looked it up once from an old textbook diagram.

- **CAPEC** (Common Attack Pattern Enumeration and Classification) catalogues attack *patterns* — the mechanics of how a technique is actually executed — and is the natural bridge between an ATT&CK technique label and the literal packet/flag pattern the model is looking at.
- **CVE / NVD** (Common Vulnerabilities and Exposures / National Vulnerability Database) catalogue specific, versioned software vulnerabilities; a stretch-goal feature is correlating a Lateral Movement or Initial Access forecast against known CVEs for the ports/services involved, which is easy to gesture at in the pitch even if not fully wired into the demo.

Part 4 — The CyberOracle Architecture

This section formalises and tightens the architecture already sketched on the team's Proposed Solution and Technical Approach slides, aligns it explicitly against every requirement identified in Part 1, and adds the two pieces the brief makes mandatory but the current slides do not yet show: a baseline comparison and a generalisation test.

4.1 The Pitch, In One Paragraph

CyberOracle represents a monitored network as a time-evolving graph, learns a Transformer-and-GNN world model of how that graph's state transitions from one observation window to the next, and rolls that model forward K steps to forecast the probability and MITRE ATT&CK stage of an infiltration before it completes — explaining every forecast with SHAP and attention, and benchmarked against a logistic-regression baseline to prove the temporal modelling is actually earning its complexity.

4.2 Formal Problem Formulation

Let $S^t \in \mathbb{R}^D$ denote the state of the network at observation window t — a vector (or graph) encoding active flows, protocol-flag distributions, port-activity patterns and packet-timing statistics for that window. The learning objective is the conditional transition distribution:

$P(S^{t+1} \mid S^t, S^{t-1}, \dots, S^{t-W})$ — the probability of the next state given a historical window of W past states.

Forward simulation (rolling this transition model out autoregressively for K steps) produces a trajectory $\tau = \{S^{t+1}, S^{t+2}, \dots, S^{t+K}\}$, from which an *infiltration likelihood* $\Lambda(t, K)$ and a most-probable MITRE ATT&CK stage are derived. This is the exact mathematical object the brief is asking for when it says “represent network state ... learn state-transition dynamics ... forecast future network states.” Everything in 4.3–4.4 is a concrete instantiation of this formulation.

4.3 End-to-End Pipeline

PCAP / NetFlow CSV → Feature Extraction (flow + packet) → Time-Windowed Graph Build

Graph Build → GNN Spatial Encoder → Per-Window Embedding

Embedding Sequence → Temporal Transformer / LSTM (World Model) → Latent Trajectory

Latent Trajectory → K-Step Rollout Head → Infiltration Probability + ATT&CK Stage

Prediction → SHAP + Attention Explainer → Offline Dashboard (Streamlit / Flask / CLI)

4.4 Component-by-Component

Component	What it does	Concrete implementation choice
Dual-Tier Ingestor	Parses raw PCAP and/or NetFlow CSV into structured flow- and packet-level features for each window	Scapy or PyShark for PCAP; pandas for CIC-IDS/CTU-13 CSVs; CICFlowMeter-style feature schema as the normalised internal format
Graph State Constructor	Builds one graph per observation window: hosts as nodes, flows as edges carrying the feature vector	NetworkX during prototyping; PyTorch Geometric `Data`/`HeteroData` objects for training
GNN Spatial Encoder	Aggregates each host's local connectivity into a node embedding, then pools into one per-window graph embedding	GraphSAGE or GATv2 via PyTorch Geometric — GATv2 preferred if time allows, since its attention weights double as an explainability signal
Temporal World Model (M-module)	Learns $P(S_{t+1}^w S_t^w, \dots)$ across the sequence of per-window graph embeddings	LSTM baseline first; Temporal Transformer (multi-head self-attention over the window sequence) as the stretch upgrade
K-Step Rollout & Stage Head	Autoregressively unrolls the world model K windows forward; a classification head maps the resulting latent trajectory to an infiltration probability curve and a MITRE ATT&CK stage	Small MLP heads on top of the final Transformer/LSTM hidden state; K chosen empirically (start with K=5 windows)
Explainability Module	Produces a feature-attribution and/or attention-weight explanation for every forecast	SHAP (KernelSHAP/TreeSHAP against the rollout head) + raw attention-weight visualisation from the Transformer/GATv2 layers
Baseline Model	A logistic-regression classifier on the identical feature set, trained and evaluated the same way	scikit-learn `LogisticRegression`; mandatory per the brief's Expected Solution section — see 4.5
Offline Dashboard	Accepts a PCAP/CSV, runs inference, displays the risk timeline, flagged flows and ATT&CK annotations — no cloud calls	Streamlit for the fastest path to a working, screenshot-ready demo within the 5-slide / 2-page / 2-minute constraints

4.5 The Mandatory Baseline — Do Not Skip This

Why this table alone can decide the evaluation

The brief does not ask the team to claim the world model is better — it asks the team to **prove it, on a like-for-like feature set**, against a named baseline. A submission with an F1 score and no comparison point is an unverifiable assertion; a submission with this table is a result.

Metric	Logistic Regression baseline (same features)	CyberOracle world model	Why it matters
Macro F1-score	to be measured	to be measured, target: visibly higher	Harmonic precision/recall mean — robust to the class imbalance every one of these datasets has
Precision / Recall	to be measured	to be measured	Shows whether the world model's gain, if any, comes from fewer false alarms or from catching more true attacks
False Positive Rate	to be measured	to be measured, target: lower	The single number that determines whether a SOC would tolerate the tool — alert fatigue is the #1 reason predictive tools get switched off
Lookahead lead time	not applicable (baseline has no forecast horizon)	to be measured, in seconds/minutes before the labelled attack timestamp	The metric that is unique to the world-model framing and cannot be produced by the baseline at all — the strongest single differentiator to put on a slide

Practically: train the logistic-regression baseline on day one, before the world model even exists (Part 10, Phase 3). It takes an afternoon, it is the PS's own explicit ask, and it gives the team a working demo and a real number to report from the very first week — insurance against a late, incomplete build.

4.6 The Generalisation Test — Turning a PS Requirement Into a Slide

The brief specifically warns against a model that “merely memorises signatures.” The cheapest, most convincing way to answer that in one slide is a **held-out attack family test**: train on all attack types in the chosen dataset except one (for example, withhold all Botnet-labelled flows from CSE-CIC-IDS2018), then evaluate detection/forecast performance on the withheld family alone. A model that still scores meaningfully above chance on an attack type it never saw during training is direct, falsifiable evidence of generalisation — far more convincing to a technical judge than an accuracy number on the training distribution. A second, stronger version of the same idea is a **cross-dataset test**: train on CSE-CIC-IDS2018 and evaluate on CTU-13's botnet captures, which differ in collection environment, feature extraction pipeline and time period. Either result, reported honestly (including if performance drops, with a one-line explanation of why), reads as more credible than a suspiciously perfect same-distribution number.

Part 5 — Frontend ↔ Backend: Exactly How Data Moves

This is the most commonly fumbled question in the viva (“where is the AI actually running?”), because it is the point where a hackathon prototype's simplifications become visible if the team hasn't rehearsed a precise answer. This section gives three honest layers: what exists today, what this submission round actually needs, and what a production system would look like — and is explicit about which layer is which at every point.

5.1 Layer 1 — What Exists Today (the React Prototype)

The deployed prototype at the team's Netlify link is a client-side React 19 + Vite + Tailwind single-page application. There is no backend: `src/data/attackScenarios.js` acts as an in-memory mock API, and a `useEffect`-driven timer advances a `stepIndex` roughly every 1.2 seconds, calling `getStepData()` to pull the next pre-computed slice of CIC-IDS2018-derived scenario data and re-render the risk graph, MITRE stage, network topology and SHAP card. This is a legitimate and common hackathon technique — it demonstrates the complete intended user experience without needing a live ML backend built first — but it must be described accurately: this is a **UX and interaction-design prototype**, not a system currently doing machine-learning inference. Claiming otherwise under questioning is the single fastest way to lose credibility with a technical panel.

The safe, honest one-liner for “is this live?”

“The deployed link demonstrates the full interaction design using pre-computed, CIC-IDS2018-derived scenario data on a timer. The machine-learning pipeline — feature extraction, the world model, SHAP — is being built as a separate Python service, shown here [switch to the Streamlit/notebook demo], and this round's job is to wire the two together.”

5.2 Layer 2 — What This Submission Round Actually Needs

Re-read the Expected Solution wording from Part 1.3: it explicitly accepts “**Streamlit, Flask web app, or CLI**” and requires the whole thing to “**run fully offline without cloud API dependencies.**” That is a materially lighter bar than the polished React/FastAPI/WebSocket product vision — and hitting it well is worth more marks than a partially-wired fancy stack. The pragmatic target architecture for the next 25 days is therefore a **single-process Python application**:

Streamlit UI (file upload widget) → Python inference module (same process) → Trained model + SHAP explainer (loaded from disk)

Concretely: a Streamlit app with a file-uploader for a PCAP or flow CSV; on upload, it calls the same feature-extraction, graph-construction and model-inference functions directly (no HTTP hop needed — Streamlit runs the Python callback in-process), then renders the results — a risk-over-time line chart, a table of the top flagged flows, the predicted ATT&CK stage, and a SHAP bar chart — using Streamlit's built-in chart components. This satisfies every line of the Expected Solution section with the least moving parts, which matters enormously with 25 days on the clock.

Recommended positioning for the pitch: keep the React dashboard as the *product vision* (it is genuinely more polished, and “we’ve also designed the production-grade interface” is a strong slide), while making the Streamlit app the *actual submitted, working demo* that satisfies the letter of the brief. Two real artefacts, honestly labelled, beat one over-claimed one.

5.3 Data Flow, Step by Step (Layer 2 Build)

6. User opens the Streamlit app and uploads a `.pcap` file or a flow CSV (or selects a bundled sample file for a guaranteed-good demo).
7. The ingestion function detects the file type; PCAPs are parsed with PyShark/Scapy, CSVs are loaded with pandas directly.
8. The feature-extraction function computes the flow-level and packet-level feature set described in Part 1.4 for each time window, using a fixed window size (e.g. 30 seconds) and a fixed history length W .
9. The graph-construction function builds one PyTorch Geometric graph object per window (hosts as nodes, flows as edges).
10. The trained GNN + temporal model (loaded once at app start-up from a saved checkpoint) is run in inference mode across the window sequence, producing the K -step-ahead infiltration probability curve and the predicted ATT&CK stage.
11. The SHAP/attention explainer runs against the same forward pass to produce a feature-attribution list for the highest-risk window.
12. Streamlit renders: (a) a risk-over-time chart with the K -step forecast extending past the current window, (b) a table of the flows driving the current risk score, (c) the predicted ATT&CK stage badge, and (d) the SHAP bar chart — all in the same page, all computed locally, nothing leaving the machine.

5.4 Layer 3 — The Production / Enterprise Vision (for the roadmap slide)

This is the architecture already sketched in the team's existing guides (React + FastAPI + WebSockets + Kafka), correctly positioned as *where this goes after the hackathon*, not as a claim about the current prototype. It answers “how would this actually scale to a real SOC” without overstating what has been built in three weeks.

Layer	Technology	Responsibility
Ingestion	Kafka event stream (or a lightweight message queue for a smaller deployment) fed by mirror/TAP-port packet brokers	Buffer high-throughput live packet/flow telemetry so the inference service is never blocked by ingestion spikes

Layer	Technology	Responsibility
Inference service	Python (FastAPI) hosting the trained GNN + temporal model	Stateless request handling; loads the model once, serves inference for each new window
Streaming channel	WebSocket (wss://) from FastAPI to the React dashboard	Pushes new risk scores, ATT&CK stage changes and flagged flows to the UI the moment they are computed, without polling
Discrete-action REST endpoints	FastAPI `POST /api/ingest/pcap`, `POST /api/mitigate`, `GET /api/analytics`	One-shot operations: uploading a capture file, triggering a mitigation action, or pulling historical analytics — REST is the right tool here because these are request/response, not continuous streams
Storage	PostgreSQL / TimescaleDB	Time-series storage for historical risk scores and incident logs, feeding the Analytics & History view
Presentation	React 19 + Vite + Tailwind + Recharts + Framer Motion (the existing prototype's stack)	The operator-facing dashboard — unchanged in spirit from the current prototype, just now fed by real data instead of a timer

Example WebSocket message (backend → frontend, production vision)

```
{ "type": "risk_update", "window_id": 4821, "timestamp": "2026-09-14T10:32:00Z",
  "infiltration_probability": 0.71, "attck_stage": "Lateral Movement", "top_features": [{"feature":
    "dst_port_445_rate", "contribution": 0.29}, {"feature": "syn_flag_variance", "contribution": 0.21}],
  "flagged_flows": ["10.0.4.12:51422 -> 10.0.1.7:445"] }
```

Example REST contract (production vision)

Endpoint	Method	Purpose	Request → Response (shape)
/api/ingest/pcap	POST	Upload a capture file for offline analysis	multipart file → { job_id, status }
/api/analytics	GET	Historical risk trend and past incidents for the dashboard's Analytics view	query params (time range) → array of window summaries
/api/mitigate	POST	Trigger (or, in the demo, simulate) an isolation/blocking action on a flagged host	{ host_ip, action } → { status, applied_rule }
/ws/live	WebSocket	Continuous stream of risk_update messages as new windows are scored	server-push JSON messages, shape as above

5.5 Security of the Tool Itself

A cybersecurity tool that is itself insecure is an easy, embarrassing line of questioning to walk into — and a strong, mature answer if pre-empted. Worth stating explicitly in the Architecture Document and being ready to expand on live:

- The offline/air-gapped design is not only a PS requirement but a security property in its own right: no outbound calls means no telemetry-exfiltration surface and no dependency on an external API's availability or trustworthiness.
- Model checkpoints and configuration should be integrity-checked (a simple checksum is enough for the hackathon; a signed-artifact pipeline is the production answer) so a compromised host cannot silently swap in a manipulated model.
- The dashboard should run with least-privilege access to the network it monitors — read-only off a mirror/TAP port, never inline on the traffic path, so a bug in the analysis tool itself cannot become a new point of failure or a lever an attacker can use to disrupt legitimate traffic.

Part 6 — Who Benefits: Stakeholders, Users, Deployment

“Who benefits” is not a single answer — a strong Impact & Benefits slide separates the institutional sponsor, the sectors ultimately protected, the individual analyst who touches the tool daily, and the citizen who never sees it but is protected by it. Treating these as one blob is the most common weakness in student pitches for government problem statements.

6.1 The Sponsor: NTRO

NTRO's direct interest is a proactive-defence capability it can point to and, if the idea progresses past the Grand Finale, potentially pilot or adapt for its own technical-intelligence mandate. What NTRO gets from a strong submission specifically: a working demonstration that a small student team can already show measurable lead-time and generalisation gains from world-model-style forecasting on public data — de-risking the case for further investment in the approach.

6.2 The Strategic Beneficiary: NCIIPC and the Six Critical Sectors

Grounding “protects Critical Information Infrastructure” in NCIIPC's own named sectors, with a concrete example of what an early forecast buys each one, is far more convincing than the phrase alone:

CII Sector	What an early, explainable forecast buys this sector
Power & Energy	Minutes of extra lead time before a reconnaissance-to-lateral-movement chain reaches SCADA/OT-adjacent IT systems — directly relevant given documented power-grid targeting incidents (Part 2.3)
Banking, Financial Services & Insurance (BFSI)	Earlier isolation of a credential-stuffing or lateral-movement chain before it reaches core banking systems, where the CERT-In-led CSIRT-Fin already coordinates sector-wide response
Telecom	Earlier visibility into reconnaissance against carrier-grade infrastructure, whose disruption cascades into every other sector that depends on connectivity
Transport	Protection for increasingly networked signalling, ticketing and logistics-coordination systems
Government	A CERT-In-aligned, explainable early-warning layer for e-governance platforms already mandated to follow the June 2023 information-security guidelines
Strategic & Public Enterprises	Defence-in-depth for PSU networks that are both high-value targets and, per the brief's own framing, environments where an offline, air-gapped-capable tool is often the only deployable option

6.3 The Direct Users — Personas

A tool is only as good as the person actually operating it at 2am. These are the personas to keep in mind when justifying every UX and explainability decision:

Persona	Goal	What frustrates them about existing tools	What CyberOracle gives them
L1 SOC Analyst (triage)	Decide, in seconds, whether an alert deserves escalation	Alert fatigue — hundreds of low-context alerts per shift from signature-based tools	A single risk-timeline view with a plain-language ATT&CK stage label instead of a raw signature ID
L2/L3 Analyst (investigation)	Understand why something is flagged well enough to act on it	“Black-box” ML scores with no feature-level justification	SHAP + attention output naming the exact ports/flags/flows driving the score
Threat Hunter	Find slow, deliberately-evasive activity that stays under rule thresholds	Signature and static-threshold tools miss low-and-slow reconnaissance by design	Sequence/timing-aware modelling built specifically to catch what per-flow thresholds miss
CISO / SOC Manager	Justify security spend and demonstrate measurable risk reduction upward	Vague “AI-powered” marketing claims with no benchmarked numbers	A reported lead-time and F1/FPR improvement over a transparent logistic-regression baseline
Compliance / Audit (NCIIPC-aligned)	Demonstrate CERT-In/NCIIPC-gui deline-aligned monitoring with an auditable trail	Opaque commercial tools that cannot be inspected or self-hosted for sovereignty/audit reasons	A fully offline, open-source, explainable pipeline whose every decision rule can be inspected

6.4 Secondary Beneficiaries

- **Citizens** who depend on the power, banking, telecom and government services in the sectors above — the ultimate, indirect but largest beneficiary group, and the honest answer if a judge asks “how many people does this affect.”
- **Smaller enterprises and MSMEs** that cannot afford an enterprise NDR licence (Part 7.2) but could self-host an open-source, offline tool built on public benchmark data.
- **The research community**, if the team's held-out-attack-family and cross-dataset generalisation results (Section 4.6) are written up — a plausible, genuinely low-effort path to a short paper or workshop submission after the hackathon.

6.5 Deployment Topology — Where This Physically Runs

Enterprise / CII network core → Mirror or TAP port (passive tap, no inline placement) → CyberOracle ingestion + inference host → Offline analyst dashboard (on-prem SOC workstation)

Three deployment properties come directly from the brief and from NTRO/NCIIPC's own operating context, and are worth stating explicitly rather than leaving implicit:

- **On-premises, not cloud.** The brief mandates offline operation; CII operators in the sectors above frequently cannot or will not send raw traffic data off-site at all, cloud or otherwise.
- **Passive, off a mirror/TAP port — never inline.** The tool observes a copy of the traffic; it does not sit in the path packets must traverse, so a crash or bug in CyberOracle itself cannot become a new source of network downtime.
- **Runs on modest, single-machine hardware.** A laptop or single server with 8–16GB RAM (optionally a consumer GPU for faster training) is enough — consistent with the brief's “no new hardware” framing for economic feasibility, and realistic for the smaller CII operators who cannot provision a GPU cluster.

Part 7 — Why CyberOracle Wins: Novelty & Positioning

“What makes this different” is one of the two or three questions every panel asks. The strongest answer names real alternatives fairly, states exactly where each falls short of *this specific brief*, and never claims an established, well-funded commercial product is simply bad — that reads as naive to any judge who knows the space.

Approach	How it actually works	Where it stops short of PS 26153's ask
Signature/rule-based NIDS (Snort, Suricata-style)	Matches traffic against a database of known attack signatures/rules	Cannot catch novel or zero-day patterns by construction; treats each packet against a fixed rule set with no temporal reasoning at all
Static ML classifiers (Random Forest, XGBoost, MLP on flow features)	Learns a benign/malicious boundary from labelled flow-level features, one flow at a time	This is precisely the paradigm the brief critiques by name — it discards the sequencing and timing information that is the actual signature of an unfolding infiltration
Commercial anomaly-based NDR (Darktrace's Self-Learning AI, Vectra AI's attacker-behaviour models)	Builds a behavioural baseline (“pattern of life”) per environment and flags deviations from it in near-real-time, at enterprise scale, with mature SOC tooling around the alerts	Detects an anomaly once it is already occurring or has just occurred against the learned baseline; it is not built around an explicit K-step-ahead forecast of where a trajectory is heading, and (per independent reviews) commonly needs a tuning period and can still generate a meaningful volume of alerts requiring analyst interpretation
CyberOracle (this submission)	Learns explicit transition dynamics $P(S_{t+1} S_t)$ and rolls the model forward K steps to forecast a trajectory before it resolves, mapping the forecast to a named ATT&CK stage with SHAP/attention explanation, benchmarked against a transparent baseline	Positioned as a complementary early-warning and explainability layer that sits alongside existing SOC tooling — not a wholesale SIEM/NDR replacement

The fair, defensible framing for the pitch

“We are not claiming existing tools can't detect threats — modern SIEM and NDR platforms are genuinely capable of detection, correlation and investigation. CyberOracle's contribution is a predictive layer: instead of asking only ‘*did something bad just happen,*’ it asks ‘*given how this network's state has been moving, is it heading toward compromise, and how many minutes do we have.*’ That is a genuinely different question, not a claim that everything else is worse.”

7.1 What Is Genuinely Novel Here

- **Forecast-then-classify, not classify-then-alert.** The K-step rollout produces a probability curve that extends into windows that have not happened yet — the defining property the brief asks for and that most alternatives, commercial or academic, do not centre their design on.
- **A measurable lead-time metric.** Unlike accuracy or F1 alone, “minutes of warning before the labelled attack timestamp” is a metric that only a genuinely predictive system can report — and it is the number that will resonate most with a non-ML judge.
- **Explainability as a first-class, mandatory output, not a bolt-on.** The brief itself states black-box output is unacceptable; building SHAP and attention in from the start (rather than trying to explain a finished black box after the fact) is both the right engineering order and a direct match to the grading criteria.
- **Fully offline and open-source.** Directly addresses the sovereignty and auditability concerns that matter specifically for a national-security-adjacent sponsor — a foreign, closed-source, cloud-dependent commercial product is not a realistic fit for many of NCIIPC's own Critical Sectors regardless of its detection quality.

Part 8 — Feasibility, Viability and Risk

This expands the team's existing Feasibility & Viability slide with the extra rigour a 2-page Architecture Document and a technical panel will expect — named risks, honest probabilities, and mitigations that are specific enough to actually execute rather than generic reassurance.

8.1 Feasibility

Dimension	Assessment
Technical	Every component (PyTorch, PyTorch Geometric, scikit-learn, SHAP, Streamlit) is a mature, well-documented open-source library; the architecture combines established techniques (GraphSAGE/GATv2, LSTM/Transformer, SHAP) rather than inventing new ones, which is exactly the right risk profile for a 3–4 week build
Data	Runs entirely on public, labelled benchmark datasets explicitly sanctioned by the PS itself — zero data-collection lead time, and zero ambiguity about whether the data source is acceptable to the evaluators
Compute	Trains and runs inference on a single CPU or consumer GPU; no cluster, no paid cloud credits, no special hardware access that could bottleneck a student team
Skill fit	Every sub-task — feature engineering, GNN training, a Streamlit UI, SHAP integration — is well-covered by public tutorials and documentation, meaning the team is not dependent on a single member's specialised expertise for any one component

8.2 Viability

- **Market/institutional pull:** CERT-In's own Security Assurance Framework and CII operators already face a compliance-driven push toward predictive defence (Part 2.4); this is not a solution looking for a problem.
- **Policy alignment:** sits comfortably alongside existing CERT-In guidelines and NCIIPC's stated R&D-funding mandate for “innovative future technology” in CII protection — the exact phrase NCIIPC uses for the kind of work this PS is soliciting.
- **Low switching cost:** reuses telemetry (NetFlow/PCAP) that CII operators already collect for other purposes; no new sensors, no new hardware, no new licensing.
- **Scalability path:** the Part 5.4 production architecture (Kafka + FastAPI + WebSockets) is a standard, well-understood scaling pattern, not a research problem in itself — the R&D risk is concentrated entirely in the model, which is exactly where the team's effort is going.

8.3 Risk Register

Risk	Likelihood	Impact	Mitigation
Combined GNN + temporal pipeline too complex to finish and debug in the time available	Medium–High	High (no working demo)	Ship the LSTM-only, non-graph baseline first (Part 10, Phase 4); add the GNN encoder only once that path works end-to-end. Never let the more ambitious architecture block having something demoable
Severe class imbalance (attacks are a small minority of most datasets, especially CICIOT2023)	High	Medium (inflated accuracy, misleading results)	Use class-weighted loss and report macro-F1 and per-class recall, not raw accuracy; this is standard practice and easy to implement in an afternoon
Weak generalisation to attack types not seen in training	Medium	High (directly contradicts the brief's own stated requirement)	Run the held-out-attack-family test from Section 4.6 early, not as an afterthought, so there is time to react if results are weak
SOC analysts distrust an automated stage prediction; ATT&CK mapping looks heuristic to evaluators	Medium	Medium (credibility, not a technical failure)	Keep the state→ATT&CK-stage mapping as an explicit, inspectable rule/threshold set rather than another opaque model, and put SHAP/attention output directly in the interface next to every prediction
Demo fails live (file won't load, model not found, dependency conflict) during the finale or a review call	Medium	High (looks unprepared regardless of underlying work quality)	Bundle at least one known-good sample file inside the repo; pin all dependency versions in a `requirements.txt`; record the 2-minute demo video as a rehearsed backup that does not depend on a live network/venue
Team under-estimates the two new, non-PPT deliverables (README, 2-page Architecture Document)	Medium	Medium (marks lost on compliance, independent of idea quality)	Track all five deliverables from Part 1.3 on a single checklist from day one; use the Appendix's copy-ready drafts rather than writing them from scratch on deadline day

Part 9 — Impact and Benefits

The team's existing Impact & Benefits slide already uses a sound four-category structure. This section grounds each category in the specific facts from Parts 2, 6 and 7 so every bullet on that slide has a defensible, specific fact behind it if a judge pushes on “how do you know that.”

9.1 National Security — Proactive Defence

- Shifts the default posture from reactive (alert after compromise) to proactive (forecast before compromise), directly addressing the pattern documented in the AIIMS and power-grid incidents in Part 2.3, where the damaging event was preceded by a quiet build-up existing tools did not surface in time.
- Shields exactly the sectors NCIIPC has named as critical (Part 6.2), rather than a generic “enterprise security” claim.
- Maps every forecast to a MITRE ATT&CK stage using the current v19 taxonomy, giving defenders a shared, standardised vocabulary for faster inter-agency coordination — relevant given CERT-In's own sectoral CSIRT model depends on that kind of shared language.

9.2 Economic Value — Cost-Efficient by Design

- Earlier warning reduces breach dwell time, and dwell time is the single biggest driver of breach and downtime cost in every major incident-cost study — the AIIMS outage alone ran to roughly two weeks of degraded hospital operations.
- Open-source, offline stack keeps licensing and cloud cost at zero, which matters for the smaller CII operators and PSUs that cannot afford enterprise NDR contracts (Part 7 table).
- Reduces dependence on foreign-owned commercial SOC tooling for a national-security-adjacent use case — a genuine strategic-autonomy argument, not just a cost one.

9.3 Social Trust — Protecting Digital India

- Safeguards the banking, power and e-governance rails that CERT-In's own Cyber Swachhta Kendra figures show now cover 98% of India's digital population — the honest “how many people does this affect” answer is effectively the entire online population, indirectly, through the sectors it protects.
- Builds public trust in digital public services at a moment when CERT-In's own year-on-year incident count is accelerating, not plateauing (Part 2.1).
- Directly supports NTRO and NCIIPC's stated statutory mandate under Section 70A of the IT Act.

9.4 Strategic Research — A Platform, Not Just a Prototype

- The attention + SHAP explainability layer is what builds analyst trust and drives real adoption — the difference between a tool that gets switched off after two weeks of false positives and one that becomes part of daily SOC workflow.
- The world-model approach generalises beyond network security to any domain with a similar detect-early-from-a-time-evolving-graph structure (fraud rings, disinformation-network growth, epidemiological spread) — a legitimate one-line “where else this could go” answer if asked.
- The held-out-attack-family and cross-dataset results (Section 4.6), if the team chooses to write them up, are a realistic, low-effort path to a short paper — turning a hackathon build into a durable research contribution rather than a one-off demo.

Part 10 — Step-by-Step Build Plan

Today is 5 September 2026; the idea-submission deadline is **20 September 2026** — 25 days away. This plan is built backward from the five deliverables in Part 1.3 and forward from the risk register in Part 8.3: get something correct and demoable early, then layer sophistication on top of a working baseline, never the other way round.

10.1 Build Order — the One Rule

If only one instruction from this whole dossier gets followed

Build the logistic-regression baseline and a bare Streamlit demo in week one, before the GNN or Transformer exists. A team with a working, boring baseline and a partially-finished world model on 20 September has a submission. A team with a brilliant, unfinished world model and no working demo does not — however good the theory in the PPT looks.

10.2 Phase-by-Phase Technical Plan

Phase	Goal	Key tasks	Tools / libraries
0. Setup	Environment and data ready	Set up Python env; download CSE-CIC-IDS2018 (and CTU-13 for later); set up GitHub repo with the structure from the README skeleton (Appendix)	Python 3.10–3.12, conda/venv, git
1. Feature extraction	Turn raw flow CSVs / PCAPs into a clean, windowed feature matrix	Load CIC-IDS2018 CSVs with pandas; write a PCAP-parsing path with Scapy/PyShark for the packet-level features (TTL variance, TCP window size, retransmission counts); handle known NaN/Infinity artefacts (Part 3.5); define the fixed observation-window size	pandas, numpy, scapy or pyshark, scikit-learn (scalers)
2. Baseline model	A working, measured logistic-regression classifier — the PS's mandatory comparison point	Train/test split respecting time order (no shuffling across the attack timeline); report F1, precision, recall, FPR	scikit-learn
3. Graph construction	One graph per window: hosts as nodes, flows as edges	Build a NetworkX prototype first to sanity-check the structure, then port to PyTorch Geometric `Data` objects	networkx, torch, torch_geometric

Phase	Goal	Key tasks	Tools / libraries
4. World model v1 (LSTM)	The lowest-risk version of the temporal engine, over per-window graph embeddings	Train an LSTM to predict the next window's embedding / infiltration label; this is the fallback that must work before anything fancier is attempted	PyTorch
5. K-step rollout + ATT&CK mapping	Forecast K windows ahead; map the resulting state to one of the five ATT&CK stages	Autoregressive rollout function; a simple, explicit rule/threshold layer (not another opaque model) mapping predicted feature patterns to Reconnaissance / Initial Access / Lateral Movement / C2 / Exfiltration	PyTorch
6. GNN upgrade	Replace the flat embedding with a GraphSAGE/GATv2 spatial encoder	Swap in a PyTorch Geometric GraphSAGE or GATv2 layer feeding the same LSTM/Transformer; only attempt after Phase 5 works end-to-end	torch_geometric
7. Temporal Transformer (stretch)	Upgrade the LSTM to a self-attention temporal model	Multi-head attention over the window-embedding sequence; keep the LSTM version as a fallback if time runs out	PyTorch
8. Explainability	SHAP + attention output for every prediction	Wire KernelSHAP/TreeSHAP against the rollout head; extract and visualise attention weights from the Transformer/GATv2 layers	shap, matplotlib/plotly
9. Generalisation test	The held-out-attack-family and/or cross-dataset result from Section 4.6	Retrain excluding one attack family; separately, evaluate on CTU-13 after training on CIC-IDS2018; write both results down even if imperfect	same stack
10. Demo interface	A working, offline Streamlit app	File upload → inference → risk chart + flagged flows + ATT&CK badge + SHAP chart, per Part 5.3	streamlit, plotly/matplotlib
11. Packaging	All five deliverables finished and cross-checked	README, 2-page Architecture Document, 5-slide PPT, 2-minute demo video, final GitHub push	—

10.3 Calendar — Four Sprints to 20 September

Sprint 1 · 5–8 September — Data and the Baseline

- Phases 0–2 complete: environment set up, CIC-IDS2018 loaded and windowed, feature matrix built, logistic-regression baseline trained and measured.

- Deliverable check-in: the team can already show a real F1/precision/recall/FPR number by the end of this week — the single most important insurance policy against a late build.

Sprint 2 · 8–12 September — The World Model, End to End

- Phases 3–5 complete: graph construction, LSTM world model, K-step rollout and rule-based ATT&CK mapping all working together on at least one dataset.
- Deliverable check-in: a script (even command-line only) that takes a CSV/PCAP in and prints an infiltration-probability curve and predicted stage out — the functional core of the whole system exists.

Sprint 3 · 12–16 September — Sophistication and Proof

- Phases 6–9 attempted in this priority order: GNN upgrade, explainability (SHAP + attention), generalisation test, and the Transformer upgrade only if time remains.
- Deliverable check-in: the Streamlit demo (Phase 10) should be started in parallel from day one of this sprint, not left until the end — a UI is far easier to build against a working backend than to bolt on afterwards.

Sprint 4 · 16–20 September — Packaging, Not Building

- No new model work after 26 September — this window is for the README, the 2-page Architecture Document, tightening the 5-slide PPT, recording the 2-minute demo video (multiple takes, keep the cleanest), and a final end-to-end run-through on a clean machine or fresh clone of the repo to catch any “works on my laptop” issues.
- **Submit at least a day early.** Portal traffic and last-minute technical issues spike in the final hours of any hackathon deadline; treat 29 September, not 20 September, as the real deadline.

10.4 If Selected for the Grand Finale (December 2026)

The idea-submission round and the 36-hour Grand Finale build are different modes: this round rewards a credible, well-evidenced plan and a working proof-of-concept; the finale rewards speed of execution against exactly the plan already validated here. If shortlisted, the fastest path is to treat Parts 4–5 of this dossier as the finale's build spec on day one, spend the first 4–6 hours re-establishing the Sprint-1/2 baseline and world model from this submission (do not start from zero), and dedicate the back half of the 36 hours to whichever of the GNN, Transformer, generalisation-test or production-architecture stretch goals were not finished in this round — the exact backlog Part 10.3 leaves behind.

Part 11 — The Full Viva & Judge Q&A Bank

Organised by category so the team can rehearse in blocks. Each answer is written the length a judge actually wants — a confident first sentence that answers the question, then one or two sentences of substance, not a monologue. Where an answer draws on a specific fact from earlier in this dossier, the part number is given so the underlying detail can be pulled up instantly if there's a follow-up.

11.1 Conceptual — What and Why

Q: What exact problem does CyberOracle solve?

It shifts network defence from confirming a breach after the fact to forecasting the probability and stage of an infiltration before it completes, by learning how a network's state evolves over time instead of judging each flow in isolation. *(Part 1.2, 4.1)*

Q: Why is a 'world model' different from a normal classifier?

A classifier answers whether the current flow looks malicious using only the present moment; a world model learns the transition dynamics of the network's state and rolls that forward, so it can answer where the trajectory is heading several windows before any single flow looks obviously malicious. *(Part 3.1)*

Q: Why did NTRO specifically ask for a world model rather than 'better detection'?

Because the brief's own stated failure mode of current tools is that low-and-slow reconnaissance, credential attempts and lateral movement each look harmless in isolation — only the sequence and timing across many steps reveals the pattern, which is exactly what a transition-dynamics model is built to capture and a per-flow classifier is built to discard. *(Part 1.2)*

Q: In one sentence, what makes this proactive rather than reactive?

The system outputs a probability curve that extends into future time windows, not just a verdict on the window that already happened. *(Part 4.2)*

11.2 Technical / Machine Learning Depth

Q: Why represent the network as a graph instead of a flat feature table?

Network flows are inherently relational — a NetFlow record is an edge between two hosts — and a flat table throws that structure away; graph neural networks like GraphSAGE and GATv2 were built specifically to learn from exactly this edge-and-node structure, with prior work (E-GraphSAGE, Anomal-E) showing it measurably helps for network intrusion tasks. *(Part 3.2)*

Q: Why LSTM/Transformer instead of a simple moving-average or rule-based trend?

A moving average assumes the signal is roughly linear and memoryless; attacks unfold as structured multi-stage sequences (recon, then access, then lateral movement), and a learned sequence model can capture that structure instead of just smoothing noise. *(Part 3.3)*

Q: How exactly does the K-step rollout work?

The temporal model's hidden state at the current window is fed back into itself autoregressively — its own predicted next state becomes the input for predicting the state after that — for K iterations, producing a trajectory of K future states that a small head then converts into an infiltration probability curve and an ATT&CK stage. (Part 4.2, 4.3)

Q: How do you prevent the model from just memorising the training attacks?

Two ways: a strictly time-ordered train/test split so no future information leaks backward, and the held-out-attack-family / cross-dataset generalisation test in Section 4.6, which specifically measures performance on attack patterns the model never saw in training. (Part 4.6)

Q: What is your evaluation metric, and why not just accuracy?

Macro-F1, precision, recall and false-positive rate, because every one of the sanctioned datasets is heavily class-imbalanced (attacks are a small minority) — a model that always predicts 'benign' can score over 90% raw accuracy while being useless, so accuracy alone is actively misleading here. (Part 4.5, 8.3)

Q: Why is the logistic-regression baseline in your results?

The brief explicitly requires it: without a transparent baseline on the identical feature set, a claim that the world model's temporal dynamics help is an assertion, not evidence. (Part 4.5)

Q: What's your false-positive rate, and why does that matter more than it sounds like it should?

It's the number that determines whether a SOC will actually keep the tool switched on — alert fatigue from high false-positive tools is the most commonly cited reason predictive security products get abandoned in practice, so we report it explicitly rather than burying it behind an accuracy figure. (Part 4.5, 6.3)

Q: How did you choose the observation-window size and the value of K?

Empirically, against the attack-timeline annotations in the training dataset — the window needs to be short enough to catch fast attacks and long enough to give the flow-level aggregate features (like a SYN flood) statistical meaning; K is chosen to target a lead time of minutes, in line with the forecasting-horizon framing in the brief.

11.3 Data and Datasets

Q: Why CIC-IDS2018 as the primary dataset?

It's on NTRO's own sanctioned list, has clean flow CSVs with documented attack-timeline labels (needed for supervised transition-dynamics training), and is the most widely used modern NIDS benchmark, which makes any reported numbers directly comparable to published literature. (Part 3.5)

Q: Why also use CTU-13 if CIC-IDS2018 already works?

As the cross-dataset generalisation check — CTU-13 is real botnet traffic captured in a different environment with a different feature pipeline, so performing above chance there is much stronger evidence of genuine generalisation than a second run on more CIC-IDS2018 data. *(Part 1.5, 4.6)*

Q: What are the known weaknesses of these datasets, and how do you handle them?

CIC-IDS2018/2017 have documented NaN/Infinity artefacts in some pre-computed flow features that need explicit cleaning; CICIoT2023 is heavily skewed toward DDoS records and needs class-weighting if used; we address both directly in the feature-extraction step rather than ignoring them. *(Part 3.5)*

Q: Is any of this live traffic, or is it all pre-recorded benchmark data?

Pre-recorded, labelled benchmark data — which is exactly what the brief itself points teams toward, since it provides the ground-truth attack timelines needed to train the transition-dynamics model in the first place; live PCAP ingestion is supported by the same pipeline and is a natural next step post-hackathon. *(Part 1.5)*

11.4 Architecture and Engineering

Q: Is the deployed prototype link actually running the AI model?

No — that link demonstrates the full interaction design using pre-computed scenario data on a timer, so the complete user experience can be shown without gating it on the ML pipeline being finished first; the model itself runs in the separate Python/Streamlit pipeline shown in the working demo. *(Part 5.1)*

Q: How do the frontend and backend actually communicate?

In this submission's working demo, there is no network hop at all — Streamlit runs the Python inference code in the same process as the UI, which is simpler and still fully satisfies the brief's own accepted interface types; the roadmap architecture separates them properly, with FastAPI serving REST endpoints for discrete actions and a WebSocket stream pushing continuous risk updates to a React dashboard. *(Part 5.2, 5.4)*

Q: Why Streamlit for the actual submitted demo instead of the fancier React app?

Because the brief explicitly accepts Streamlit/Flask/CLI and requires the whole thing to run fully offline — it gets us to a fully compliant, working, screenshot- and video-ready demo with the fewest moving parts in the time available, while the React app is kept and shown separately as the production-grade vision. *(Part 5.2)*

Q: What would you build next, if you had one more month?

Wire the trained model into the FastAPI + WebSocket + React production architecture from Part 5.4, add live PCAP ingestion instead of only batch file upload, and extend the generalisation testing to the LANL and DARPA datasets for authentication- and provenance-based attack chains, not just NetFlow-based ones. *(Part 5.4, 10.4)*

11.5 Explainability

Q: What is SHAP, in one sentence?

A game-theoretic method that fairly splits a prediction's score across the input features that produced it, so each feature gets a signed contribution number that provably sums to the model's actual output. (Part 3.6)

Q: Why both SHAP and attention weights — isn't one enough?

They explain different things: SHAP explains which features drove one specific prediction; attention shows which past time-windows or graph nodes the model itself weighted most heavily, which is a structural/temporal explanation SHAP doesn't provide — together they let an analyst ask both 'what' and 'when'. (Part 3.6)

Q: Why does explainability matter this much for this specific brief?

The brief states outright that a black-box output with no interpretability is not acceptable, and separately, SOC analysts documented in our own user research (Part 6.3) will not act on a score they can't audit — it's both a grading requirement and a real adoption blocker if skipped.

11.6 Comparison and Competitive Positioning

Q: Why not just use Darktrace or a similar commercial tool?

Those tools build a behavioural baseline and flag deviations from it, which is valuable but fundamentally reactive to what's already happening against that baseline; they're also closed-source and cloud-adjacent in their most capable tiers, which conflicts with the brief's explicit offline, auditable, sovereign-deployment requirement for national-security-adjacent networks. (Part 7)

Q: What makes this better than Splunk or a standard SIEM?

We're not claiming SIEMs can't detect or correlate events — they can. CyberOracle adds a predictive layer underneath: forecasting where a trajectory is heading, in ATT&CK-stage terms, before the SIEM's own rules would fire on the completed event. (Part 7)

Q: Has anyone published something like this already?

Yes, and we cite them rather than hide from it — StageFinder (Phan & Bauschert, 2026) does GNN-plus-LSTM attack-stage estimation on DARPA data, and GraphIDS (2025) combines E-GraphSAGE with a Transformer on NetFlow benchmarks; our contribution is applying and evaluating this world-model framing specifically against PS 26153's dual flow-and-packet feature requirement and its mandatory baseline/generalisation tests. (Part 3.4)

11.7 Deployment, Ethics and Honesty Questions

Q: Where would this actually be deployed?

On-premises, passively, off a mirror or TAP port on an enterprise or Critical Information Infrastructure network — never inline on the traffic path — so it can be air-gapped where required and never becomes a new point of network failure itself. (Part 6.5)

Q: What happens if the model is wrong — false alarm or missed attack?

A false positive costs an analyst a few minutes of review, which is why we report and optimise for false-positive rate explicitly rather than only recall; a missed attack is why CyberOracle is positioned as an additional early-warning layer alongside existing SIEM/NDR tooling, not a sole line of defence.

Q: Could this tool itself be attacked or manipulated?

It's designed passively and read-only against the traffic it monitors, and model checkpoints should be integrity-checked so a compromised host can't silently swap in a manipulated model — a point we address directly rather than waiting to be asked. *(Part 5.5)*

Q: What is your current biggest limitation, honestly?

The demo currently runs on offline benchmark datasets rather than a live production network feed, and the GNN/Transformer upgrade over the LSTM baseline is still in progress — both are the explicit next steps in our build plan, not things we're pretending are finished. *(Part 10.3, 10.4)*

Part 12 — Glossary and References

12.1 Glossary

Term	Meaning
World Model	An AI architecture that learns an internal, causal simulation of how an environment's state evolves, so it can predict future states rather than only classify the present one (Part 3.1)
State vector S_t	The structured feature vector or graph encoding a network's condition at observation window t
Transition dynamics $P(S_{t+1} S_t)$	The learned probability distribution over the next network state given the current (and recent past) state
K-step rollout	Running the transition model forward K times in a row, using each predicted state as the input for predicting the next, to produce a multi-step-ahead forecast
Infiltration likelihood $\Lambda(t,K)$	The forecasted probability, at time t , that the network is on a trajectory toward compromise within the next K windows
GNN (Graph Neural Network)	A neural network architecture that learns directly from graph-structured data (nodes and edges) rather than flat feature tables
GraphSAGE / GATv2	Two widely-used GNN layer designs; GraphSAGE aggregates neighbourhood information via sampling, GATv2 additionally learns attention weights over neighbours, which double as an explainability signal
E-GraphSAGE	A variant of GraphSAGE adapted to produce edge embeddings (rather than node embeddings), matching the fact that a network flow is naturally an edge between two hosts
LSTM (Long Short-Term Memory)	A recurrent neural network variant designed to learn long-range temporal dependencies in sequential data
Temporal Transformer	A Transformer architecture (self-attention) applied across a sequence of time-windowed observations rather than, e.g., words in a sentence
SHAP (SHapley Additive exPlanations)	A game-theoretic method (Lundberg & Lee, 2017) for attributing a model's prediction fairly across its input features
MITRE ATT&CK	A publicly maintained knowledge base of adversary tactics and techniques, organised into stages (tactics) such as Reconnaissance, Initial Access, Lateral Movement, Command & Control and Exfiltration

Term	Meaning
CAPEC	Common Attack Pattern Enumeration and Classification — a MITRE catalogue of attack execution patterns, complementary to ATT&CK
CVE / NVD	Common Vulnerabilities and Exposures / National Vulnerability Database — the standard catalogue of specific, versioned software vulnerabilities
NetFlow / IPFIX	Standardised protocols for exporting aggregate statistics (source/destination, ports, bytes, duration) about IP traffic flows, without capturing full packet payloads
PCAP	Packet Capture — a full, granular record of raw network packets, including headers and (unless filtered) payload
TTL (Time-To-Live)	An IP header field indicating how many more hops a packet may traverse before being discarded; its variance across a session is a useful anomaly feature
IAT (Inter-Arrival Time)	The time gap between consecutive packets or flows; its statistics (mean, variance) can reveal scanning or beaconing behaviour invisible to volume-based features alone
SOC (Security Operations Centre)	The team and facility responsible for continuously monitoring and responding to security events in an organisation
NDR (Network Detection and Response)	A category of commercial security product (e.g. Darktrace, Vectra AI) that detects threats from network traffic patterns, typically via behavioural baselining
SIEM (Security Information and Event Management)	A platform that aggregates and correlates security logs/events across an organisation for detection and investigation (e.g. Splunk, Microsoft Sentinel)
CII (Critical Information Infrastructure)	Computer resources whose incapacitation would have a debilitating impact on national security, economy, public health or safety, as defined under Section 70A of India's IT Act, 2000
NTRO	National Technical Research Organisation — India's technical-intelligence agency and the sponsor of this problem statement
NCIIPC	National Critical Information Infrastructure Protection Centre — the national nodal agency (a unit of NTRO) for protecting India's CII
CERT-In	Indian Computer Emergency Response Team — India's national incident-response agency under Section 70B of the IT Act, 2000

12.2 References

Formatted for direct reuse on the Research & References slide. Entries are grouped by category; within each, ordered by relevance to the pitch.

Foundational theory: World Models

- D. Ha and J. Schmidhuber, “World Models,” arXiv:1803.10122, 2018.
- D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering Diverse Domains through World Models,” arXiv:2301.04104, 2023.
- Y. LeCun, “A Path Towards Autonomous Machine Intelligence,” OpenReview position paper, 2022.

Graph learning for network intrusion detection

- W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, “E-GraphSAGE: A Graph Neural Network based Intrusion Detection System for IoT,” IEEE/IFIP NOMS, 2022.
- E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, “Anomal-E: A Self-Supervised Network Intrusion Detection System based on Graph Neural Networks,” Knowledge-Based Systems, 2022.
- A. Guerra et al., “GraphIDS: A Graph-based Intrusion Detection System,” 2025 (Transformer masked-autoencoder + inductive E-GraphSAGE encoder).

Directly on-topic 2026 literature (attack-stage / alert prediction)

- T. V. Phan and T. Bauschert, “Learning the APT Kill Chain: Temporal Reasoning over Provenance Data for Attack Stage Estimation,” arXiv:2603.07560, 2026 (accepted, IEEE ICC 2026).
- “BiTA: Bidirectional Gated Recurrent Unit-Transformer Aggregator in a Temporal Graph Network Framework for Alert Prediction in Computer Networks,” arXiv:2604.22781, 2026.

Explainability

- S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

Benchmark datasets

- I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), 2018 (CIC-IDS2017 / CSE-CIC-IDS2018).
- N. Moustafa and J. Slay, “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems,” Military Communications and Information Systems Conference (MilCIS), 2015.
- S. Garcia, M. Grill, J. Stiborek, and A. Zunino, “An Empirical Comparison of Botnet Detection Methods,” Computers & Security, 2014 (CTU-13 dataset).
- E. C. P. Neto et al., “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” Sensors, 2023.

Official / institutional sources

- Smart India Hackathon 2026 portal, Problem Statement 26153 listing, sih.gov.in (accessed September 2026).
- National Critical Information Infrastructure Protection Centre, nciipc.gov.in — mandate, Critical Sectors and functions (accessed September 2026).
- Indian Computer Emergency Response Team (CERT-In) annual and 2025 incident statistics, cert-in.org.in (accessed September 2026).
- MITRE ATT&CK®, Enterprise Matrix, version 19 release notes, attack.mitre.org (April 2026).

Appendix

A.1 — One-Page Cheat Sheet

Read this once immediately before walking into the room or joining the call.

If asked...	Say this
"What does it do?"	Forecasts network attacks before they complete, by learning how the network's state evolves over time instead of judging one flow at a time.
"Why is that hard / novel?"	Per-flow classifiers throw away sequencing and timing — the actual signature of an infiltration in progress. A world model keeps it.
"What's the architecture?"	Flow+packet features → per-window graph (GraphSAGE/GATv2) → temporal model (LSTM/Transformer) → K-step rollout → ATT&CK stage + SHAP/attention explanation.
"What data?"	CSE-CIC-IDS2018 primary, CTU-13 for cross-dataset generalisation — both explicitly sanctioned by NTRO's own brief.
"How do you prove it's better?"	Benchmarked against a logistic-regression baseline on the same features: F1, precision, recall, false-positive rate, plus a lead-time number the baseline can't produce at all.
"How do you prove it generalises?"	Held-out-attack-family test and a cross-dataset (CIC-IDS2018→CTU-13) test — see Part 4.6.
"Is the deployed link live AI?"	No — it's a UX prototype on pre-computed data. The Python/Streamlit pipeline is the real ML demo. Say this plainly if asked; don't hedge.
"Who benefits?"	NTRO/NCIIPC institutionally; SOC analysts across Power, BFSI, Telecom, Transport, Government and Strategic sectors day-to-day; ultimately every citizen who depends on those services.
"Where does it run?"	On-prem, passively off a mirror/TAP port, fully offline — never inline on the traffic path.
"What's your biggest limitation right now?"	Say it plainly: which of GNN / Transformer / generalisation test is still in progress, per the current build status. Honesty here reads as strength, not weakness.

A.2 — PPT Tightening Notes (Slide by Slide)

The existing six-slide deck already follows the correct SIH template. These are surgical upgrades, not a rewrite — each ties a specific fact from this dossier to a specific slide so the deck and the dossier tell one consistent story.

Slide 1 — Title Page

- Fine as-is. Fill in the Team ID field before submission — currently blank.

Slide 2 — Proposed Solution

- Strong structure already. Add one small callout box or footnote naming the mandatory logistic-regression baseline (Part 4.5) — its absence is the first thing a technical judge will probe for, so pre-empting it costs one line and removes an easy objection.
- Consider adding “generalises to unseen attack families (held-out test)” as a fifth bullet under Key AI Mechanisms — it directly answers the brief's own stated concern about memorisation.

Slide 3 — Technical Approach

- Keep the System Architecture and Technical Stack panels. Add CTU-13 next to CIC-IDS-2018 in the Datasets box, labelled “generalisation check” — shows the dataset choice is deliberate, not just “whatever was easiest to find.”
- The prototype link is a strong asset — keep it, but be ready to describe it exactly as Part 5.1 frames it if asked live.

Slide 4 — Feasibility and Viability

- Already well-structured. Swap the generic “mandatory predictive-defence push under CERT-In/NCIIPC norms” line for the specific 2025 CERT-In figure (29.44 lakh incidents, up from 20.41 lakh in 2024, Part 2.1) — a real number lands harder than a policy phrase.

Slide 5 — Impact and Benefits

- Keep the four-quadrant structure. Under Social Trust, the CERT-In Cyber Swachhta Kendra “98% of India's digital population” figure (Part 9.3) is a stronger, more specific claim than “builds public trust.”

Slide 6 — Research and References

- **Fix reference 1** — the author list rendered incorrectly in the current PDF export (a garbled repeated-initials string). Part 12.2 has the clean, correctly-formatted reference list, including two additional, very recent (2026) papers — StageFinder and BiTA — that are not yet on the slide and will read as evidence of genuinely current research.

A.3 — Copy-Ready: 2-Page Architecture Document

How to use this section

Everything from the heading below to the start of A.4 is written to be lifted directly into its own file (as “CyberOracle_Architecture.pdf” or similar) and submitted as deliverable #3 from Part 1.3. It is deliberately dense and written at roughly two printed pages — resist the urge to expand it; the brief caps it at two pages for a reason, and a shorter, denser document reads as more confident than a padded one.

CyberOracle — Architecture Document

Problem Statement 26153 · Team Precognix · National Technical Research Organisation

1. Approach

CyberOracle is a world-model-based forecasting system for network attacks. Instead of classifying individual flows as benign or malicious, it models the network as a time-evolving graph and learns the transition dynamics $P(S_{t+1} | S_t)$ governing how that graph's state changes from one observation window to the next. Forward-simulating this model for K windows produces an infiltration-probability trajectory and a predicted MITRE ATT&CK stage, both accompanied by a SHAP/attention-based explanation.

2. System Architecture

Ingestion (PCAP/NetFlow) → Feature Extraction → Graph Construction

GNN Spatial Encoder → Temporal World Model → K-Step Rollout

ATT&CK Stage Mapping → SHAP + Attention Explainer → Offline Dashboard

3. Components

Component	Function	Implementation
Dual-tier ingestor	Parses PCAP and/or flow CSV into normalised flow- and packet-level features per window	Scapy/PyShark, pandas
Graph state constructor	Builds a per-window graph: hosts as nodes, flows as edges	PyTorch Geometric
GNN spatial encoder	Learns a per-window graph embedding	GraphSAGE / GATv2
Temporal world model	Learns state-transition dynamics across the window sequence	LSTM (baseline) → Temporal Transformer (target)
K-step rollout + stage head	Forecasts K windows ahead; maps to an ATT&CK stage	Autoregressive rollout + rule-based mapping layer
Explainability module	Attributes each forecast to driving features / time-steps	SHAP (Kernel/TreeSHAP) + attention-weight extraction
Baseline model	Transparent comparison point required by the brief	scikit-learn logistic regression, identical features
Demo interface	Offline, file-in / forecast-out interface	Streamlit

4. Data and Training

Trained primarily on CSE-CIC-IDS2018 (flow CSVs, attack-timeline labels for supervised transition-dynamics learning). CTU-13 is used as a structurally distinct dataset for cross-dataset generalisation testing. Training uses a strictly time-ordered split (no shuffling across the attack timeline) and class-weighted loss to address label imbalance.

5. Evaluation Plan

- Primary metrics: macro-F1, precision, recall, false-positive rate — world model vs. logistic-regression baseline, identical features.
- Forecasting-specific metric: mean lead time (seconds/minutes of warning before the labelled attack timestamp) — not producible by the baseline.
- Generalisation: held-out attack-family test (train on all-but-one attack type, evaluate on the withheld type) and cross-dataset test (train on CIC-IDS2018, evaluate on CTU-13).

6. Deployment Model

Designed for on-premises, passive deployment off a mirror/TAP port on an enterprise or Critical Information Infrastructure network — never inline on the traffic path. Runs fully offline with no cloud API dependency, on a single machine (8–16GB RAM, optional consumer GPU), consistent with air-gapped operational requirements common to CII environments.

A.4 — Copy-Ready: README.md Skeleton

Drop this into the repository root and fill in the bracketed sections as the build progresses — this is deliverable #2 from Part 1.3.

```
# CyberOracle
AI-Based Network Attack Forecasting from Network Traffic Data
Smart India Hackathon 2026 · Problem Statement 26153 · Team Precognix

## Overview
CyberOracle forecasts the probability and MITRE ATT&CK stage of a network
attack before it completes, by learning a Transformer/GNN world model of
how network state evolves – instead of classifying single flows in isolation.

## Architecture
See ARCHITECTURE.md (or docs/architecture.pdf) for the full 2-page design.
Pipeline: ingestion -> feature extraction -> graph construction ->
GNN encoder -> temporal world model -> K-step rollout -> ATT&CK mapping
-> SHAP/attention explanation -> Streamlit dashboard.

## Repository structure
data/                sample + processed data (raw datasets not committed)
src/features/        flow- and packet-level feature extraction
src/graph/           graph construction utilities
src/models/          GNN encoder, temporal world model, rollout head
src/baseline/        logistic regression baseline
src/explain/         SHAP + attention explainability
app/                 Streamlit demo application
notebooks/           exploration and evaluation notebooks
requirements.txt
```

Setup

```
python -m venv venv && source venv/bin/activate  
pip install -r requirements.txt
```

Datasets

Download CSE-CIC-IDS2018 (primary) and CTU-13 (generalisation test) from the University of New Brunswick / CTU sources and place under data/raw/. See dataset links in Part 1.5 / 3.5 of the project dossier.

Usage

1. Extract features

```
python -m src.features.extract --input data/raw/ --output data/processed/
```

2. Train baseline

```
python -m src.baseline.train --data data/processed/
```

3. Train world model

```
python -m src.models.train --data data/processed/ --config configs/world_model.yaml
```

4. Run the offline demo

```
streamlit run app/dashboard.py
```

Evaluation results

[Fill in: F1 / precision / recall / FPR for baseline vs. world model, lead-time in seconds, generalisation-test results – Part 4.5 / 4.6]

Team

Team Precognix – [names, roles]

License

[MIT / Apache-2.0 – pick one before submission]